

презентация по лабораторной работы №8: Элементы криптографии. Шифрование (кодирование) различных исходных текстов одним ключом

Эйвази Мани

06.октябрь.2026

Содержание

презентация по лабораторной работы №8:	3
Цели и задачи работы	3
Цель лабораторной работы	3
Процесс выполнения лабораторной работы	4
Подготовка рабочей директории и создание скрипта	4
Исходные данные и вычисление шифротекстов	5
Проверка дешифрования ключом и атака без ключа	6
Исходный код приложения main.py (часть 1)	8
Исходный код приложения main.py (часть 2)	9

Объяснение логики работы	10
Принцип однократного гаммирования одним ключом	10
Атака без знания ключа	10
Пошаговое восстановление по шаблону	10
Выводы по проделанной работе	11
Вывод	11



презентация по лабораторной работы №8:

Элементы криптографии. Шифрование (кодирование) различных исходных текстов одним ключом

Цели и задачи работы

Цель лабораторной работы

Освоить на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом.

Процесс выполнения лабораторной работы

Подготовка рабочей директории и создание скрипта

- Создана рабочая директория lab_08, содержащая подкаталоги program и report.
- Создан файл main.py с помощью текстового редактора nano.
- Файлу предоставлены права на исполнение командой `chmod +x main.py`.
- Выполнен запуск приложения командой `./main.py`.

```
root@localhost:~/lab_08# ls
program  report
root@localhost:~/lab_08# nano main.py
root@localhost:~/lab_08# ls
main.py  program  report
root@localhost:~/lab_08# chmod +x main.py
root@localhost:~/lab_08# ls
main.py  program  report
root@localhost:~/lab_08# ./main.py
=== Исходные данные ===
P1 = НВашисходящийот1204
P2 = ВСеверныйфилиалБанка
K = 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B 02 70 54
Длина K = 20 байт
Длина P1 = 20 байт
Длина P2 = 20 байт

=== Шифротексты ===
C1 = C8 EC D5 9F F6 A6 C6 27 7A F4 F6 D7 CA BE 11 3A 3A 80 40 60
C2 = C7 DD F2 9D EB BE DA 29 7D E4 E1 C5 CA B7 14 09 EB 5F 9A B4

=== Дешифрование ключом (проверка) ===
P1' = НВашисходящийот1204
P2' = ВСеверныйфилиалБанка

=== C1 xor C2 = P1 xor P2 ===
X = 0F 31 27 02 1D 18 1C 0E 07 10 17 12 00 09 05 33 D1 DF DA D4

=== Восстановление P2 при известном P1 (без ключа) ===
P2 (восстановленный) = ВСеверныйфилиалБанка

=== Восстановление P1 при известном P2 (без ключа) ===
P1 (восстановленный) = НВашисходящийот1204

=== Пошаговое восстановление P2 по шаблону P1 ===
Частично восстановленный P2: ВСеверныйфилиалБанка
root@localhost:~/lab_08#
```

Рис. 1 — Подготовка директории и запуск приложения

Исходные данные и вычисление шифротекстов

- Приложение вывело исходные данные:
 - P1 = 1204
 - P2 =
 - K = 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54
 - K = 20
 - P1 = 20
 - P2 = 20
- Вычислены шифротексты обоих сообщений:
 - C1 = C8 EC D5 9F F6 A6 C6 27 7A F4 F6 D7 CA BE 11 3A 3A 80 40 60
 - C2 = C7 DD F2 9D EB BE DA 29 7D E4 E1 C5 CA B7 14 09 EB 5F 9A B4

```
root@localhost:~/lab_08# ls
program report
root@localhost:~/lab_08# nano main.py
root@localhost:~/lab_08# ls
main.py program report
root@localhost:~/lab_08# chmod +x main.py
root@localhost:~/lab_08# ls
main.py program report
root@localhost:~/lab_08# ./main.py
=== Исходные данные ===
P1 = НВашисходящий1204
P2 = ВССеверныйфилиалБанка
K = 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54
Длина K = 20 байт
Длина P1 = 20 байт
Длина P2 = 20 байт

=== Шифротексты ===
C1 = C8 EC D5 9F F6 A6 C6 27 7A F4 F6 D7 CA BE 11 3A 3A 80 40 60
C2 = C7 DD F2 9D EB BE DA 29 7D E4 E1 C5 CA B7 14 09 EB 5F 9A B4

=== Дешифрование ключом (проверка) ===
P1' = НВашисходящий1204
P2' = ВССеверныйфилиалБанка

=== C1 xor C2 = P1 xor P2 ===
X = 0F 31 27 02 1D 18 1C 0E 07 10 17 12 00 09 05 33 D1 DF DA D4

=== Восстановление P2 при известном P1 (без ключа) ===
P2 (восстановленный) = ВССеверныйфилиалБанка

=== Восстановление P1 при известном P2 (без ключа) ===
P1 (восстановленный) = НВашисходящий1204

=== Пошаговое восстановление P2 по шаблону P1 ===
Частично восстановленный P2: ВССеверныйфилиалБанка
root@localhost:~/lab_08#
```

Рис. 2 — Исходные данные и вычисленные шифротексты

Проверка дешифрования ключом и атака без ключа

- Выполнена проверка дешифрования с использованием ключа:
 - $P1' = 1204$ (совпадает с исходным P1)
 - $P2' =$ (совпадает с исходным P2)
- Выполнена операция XOR двух шифротекстов:
 - $X = C1 \text{ xor } C2 = P1 \text{ xor } P2$
 - $X = 0F\ 31\ 27\ 02\ 1D\ 18\ 1C\ 0E\ 07\ 10\ 17\ 12\ 00\ 09\ 05\ 33\ D1\ DF\ DA\ D4$
- Восстановлен P2 при известном P1 (без ключа):
 - $P2 () =$
- Восстановлен P1 при известном P2 (без ключа):
 - $P1 () = 1204$
- Выполнено пошаговое восстановление P2 по шаблону P1:
 - P2:

```

root@localhost:~/lab_08# cat program
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

def xor_bytes(a: bytes, b: bytes) -> bytes:
    """Побитовый XOR двух последовательностей байт одинаковой длины."""
    return bytes(x ^ y for x, y in zip(a, b))

def to_hex(data: bytes) -> str:
    return ' '.join(f'{b:02X}' for b in data)

def text_to_bytes(text: str) -> bytes:
    return text.encode('cp1251')

def bytes_to_text(data: bytes) -> str:
    return data.decode('cp1251', errors='replace')

def main():

    P1_text = "НаВашисходящийот1204"
    P2_text = "ВСеверныйфилиалБанка"

    key_hex = "05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54"
    K = bytes(int(x, 16) for x in key_hex.split())

    P1 = text_to_bytes(P1_text)
    P2 = text_to_bytes(P2_text)

    print("=== Исходные данные ===")
    print(f"P1 = {P1_text}")
    print(f"P2 = {P2_text}")
    print(f"K = {to_hex(K)}")
    print(f"Длина K = {len(K)} байт")
    print(f"Длина P1 = {len(P1)} байт")
    print(f"Длина P2 = {len(P2)} байт")
    print()

    # --- Шифрование ---
    C1 = xor_bytes(P1, K)
    C2 = xor_bytes(P2, K)

    print("=== Шифротексты ===")
    print(f"C1 = {to_hex(C1)}")
    print(f"C2 = {to_hex(C2)}")

```

Рис. 3 — Проверка дешифрования и атака без ключа

Исходный код приложения main.py (часть 1)

- В приложении реализованы функции:
 - `xor_bytes(a, b)` — побитовый XOR двух последовательностей байтов одинаковой длины.
 - `to_hex(data)` — преобразование байтов в шестнадцатеричную строку.
 - `text_to_bytes(text)` — преобразование текста в байты с использованием кодировки CP1251.
 - `bytes_to_text(data)` — преобразование байтов в текст с использованием кодировки CP1251.
- В функции `main()` заданы исходные данные: P1, P2 и ключ K.

```
# --- Шифрование ---
C1 = xor_bytes(P1, K)
C2 = xor_bytes(P2, K)

print('=== Шифротексты ===')
print(f'C1 = {to_hex(C1)}')
print(f'C2 = {to_hex(C2)}')
print()

# --- Проверка: дешифрование ключом ---
P1_dec = xor_bytes(C1, K)
P2_dec = xor_bytes(C2, K)

print('=== Дешифрование ключом (проверка) ===')
print(f'P1' = {bytes_to_text(P1_dec)}')
print(f'P2' = {bytes_to_text(P2_dec)}')
print()

# --- Атака без ключа ---
# Если известен P1, то P2 = (C1 xor C2) xor P1
X = xor_bytes(C1, C2) # = P1 xor P2
print('=== C1 xor C2 = P1 xor P2 ===')
print(f'X = {to_hex(X)}')
print()

# Восстановим P2, зная P1
P2_recovered = xor_bytes(X, P1)
print('=== Восстановление P2 при известном P1 (без ключа) ===')
print(f'P2 (восстановленный) = {bytes_to_text(P2_recovered)}')
print()

# Восстановим P1, зная P2
P1_recovered = xor_bytes(X, P2)
print('=== Восстановление P1 при известном P2 (без ключа) ===')
print(f'P1 (восстановленный) = {bytes_to_text(P1_recovered)}')
print()

# --- Демонстрация пошагового восстановления ---
print('=== Пошаговое восстановление P2 по шаблону P1 ===')
known_P1 = "НаВашисходящийот1204"
known_bytes = text_to_bytes(known_P1)
# Восстанавливаем P2 там, где известен P1
partial = bytearray(len(P2))
for i in range(min(len(known bytes), len(X))):
```

Рис. 4 — Исходный код приложения main.py (часть 1)

Исходный код приложения main.py (часть 2)

- Выполнено шифрование обоих текстов одним ключом:
 - `C1 = xor_bytes(P1, K)`
 - `C2 = xor_bytes(P2, K)`
- Выполнена проверка дешифрования:
 - `P1_dec = xor_bytes(C1, K)`
 - `P2_dec = xor_bytes(C2, K)`
- Реализована атака без ключа:
 - `X = xor_bytes(C1, C2)` — получение `P1 xor P2`
 - `P2_recovered = xor_bytes(X, P1)` — восстановление `P2` при известном `P1`
 - `P1_recovered = xor_bytes(X, P2)` — восстановление `P1` при известном `P2`
- Реализована демонстрация пошагового восстановления `P2` по шаблону `P1`.

```
# --- Демонстрация пошагового восстановления ---
print("=== Пошаговое восстановление P2 по шаблону P1 ===")
known_P1 = "НаВашисходящий1204"
known_bytes = text_to_bytes(known_P1)
# Восстанавливаем P2 там, где известен P1
partial = bytearray(len(P2))
for i in range(min(len(known_bytes), len(X))):
    partial[i] = X[i] ^ known_bytes[i]
print(f"Частично восстановленный P2: {bytes_to_text(bytes(partial))}")

if __name__ == "__main__":
    main()
root@localhost:~/lab_08#
```

Рис. 5 — Исходный код приложения main.py (часть 2)

Объяснение логики работы

Принцип однократного гаммирования одним ключом

- Режим однократного гаммирования реализуется по формуле:
 - $C1 = P1 \oplus K$
 - $C2 = P2 \oplus K$
- Если злоумышленник знает оба шифротекста $C1$ и $C2$, он может вычислить:
 - $C1 \oplus C2 = (P1 \oplus K) \oplus (P2 \oplus K) = P1 \oplus P2$
- Ключ K при этом взаимно уничтожается, так как $K \oplus K = 0$.

Атака без знания ключа

- Если злоумышленнику известен один из открытых текстов (например, $P1$), он может восстановить второй текст:
 - $P2 = (C1 \oplus C2) \oplus P1$
- Если известен $P2$, аналогично восстанавливается $P1$:
 - $P1 = (C1 \oplus C2) \oplus P2$

Пошаговое восстановление по шаблону

- Если один из текстов является шаблоном с фиксированным форматом, злоумышленник может последовательно угадывать символы второго текста, используя известные позиции шаблона.
- На каждом шаге используется формула $P2 = X \oplus P1$, где $X = C1 \oplus C2$, а $P1$ — известные символы шаблона.
- Это позволяет значительно сократить пространство поиска и постепенно восстановить весь текст $P2$.

Выводы по проделанной работе

Вывод

В ходе лабораторной работы было освоено на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом. Разработано приложение `main.py`, позволяющее шифровать и дешифровать тексты P1 и P2 в режиме однократного гаммирования, а также реализующее атаку без знания ключа.

Продемонстрировано, что при шифровании двух различных текстов одним ключом возникает уязвимость: злоумышленник, зная оба шифротекста, может вычислить P1 XOR P2, что при знании одного из открытых текстов позволяет восстановить второй. Показано, что использование одного ключа для шифрования нескольких сообщений является небезопасным и противоречит условиям абсолютной стойкости шифра.

Полученные результаты согласуются с целью работы.